University of Cagliari

MSc programs in Computer Engineering, Cybersecurity and Artificial Intelligence – Electronic Engineering – Internet Engineering

Artificial Intelligence

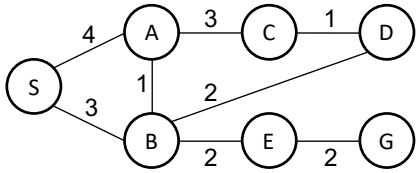
Academic Year: 2026/2027

Instructor: Giorgio Fumera

Exercises on search algorithms

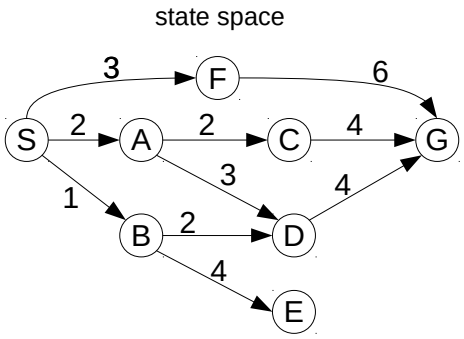
1. Consider the state space of a graph search problem shown below. States are denoted by letters. The number on each edge denotes the cost of the corresponding action (note that actions are reversible), and `S` and `G` denote respectively the initial and goal states. The table on the right shows the values of an *admissible* heuristic function (i.e., it never overestimates the minimum path cost from any given state to the goal state `G`).

   Show how the depth-first and A* search strategies solve this problem, avoiding repeated states in the same path. In case of tie, for depth-first search choose the node to expand next according to *alphabetical order*; for A* choose the node with the lowest path cost, and in case of further tie choose the shallowest node. For the sake of clarity, draw *separately* the search tree obtained after each expansion step, and *clearly* indicate the expanded node. For A* separately indicate the path cost and the value of the heuristic function of each node. For depth-first search, remove nodes with no successors (if any).



S: 5	A: 3	C: 1
D: 2	B: 2	E: 1

2. The graph in the figure below shows the state space of a search problem whose actions are *not* reversible, as indicated by the fact that the graph is *oriented* (e.g., it is possible to move from state `S` to `A`, but not vice versa). The table on the right shows the value of an admissible heuristic function, considering `G` as the goal state.



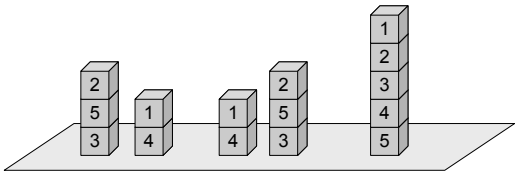
heuristic function (goal state: G)

S	A	B	C	D	E	F	G
6	4	5	2	2	8	4	0

   Show how breadth-first search, depth-first search and A* find a path from `S` to `G`, following the instructions of the previous exercise (including the tiebreaker). For breadth-first search and depth-first search show also the path cost of each node.

3. The *block's world* is a well-known toy domain used in AI for planning problems related to robots. It consists of a set of blocks of various size, shape, and color, possibly identified by a letter or a number, and placed on a table; the blocks can be stacked into one or more piles, possibly with some constraints (depending, e.g., on their size); the goal is to form a target set of piles starting from a given initial configuration. The available actions consist of picking a block on top of one of the current piles (including piles made up of a single block), and placing it either on the table (unless it was already on the table) or on top of one of the other piles.

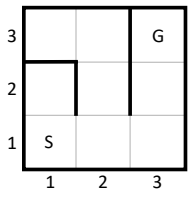
   Consider a simple version of this problem: five blocks with identical shape and size, numbered from 1 to 5. Three possible configurations are shown below. Note that the relative position of the piles does not matter, thus the configuration on the left is equivalent to the one in the middle. Every block can be either on the table or on the top of another block, so there can be one to five piles. Starting from any given configuration, the goal is to stack the blocks in a single pile, shown at the right of the figure below, by moving the *smallest* number of blocks.



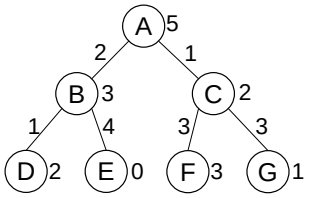
   (a) Formulate the above version of the block's world as a search problem, by *formally* defining the state space, the initial state, the goal test, the operators and the step cost.

   (b) Assume the initial configuration is the one on the left in the figure above, and consider a heuristic function defined as the number of blocks that are not in the highest pile. For instance, the heuristic function of the start state equals 2, whereas for a configuration with all five blocks on the table it equals 4. Under this setting, draw the search tree obtained after the first *four* expansion steps by A*, avoiding repeated states, and breaking ties (if any) randomly. Clearly indicate the order of expansion, as well as the path cost and the value of the heuristic function of each node.

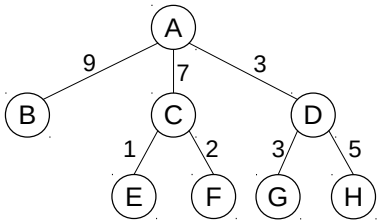
4. Find the shortest way from square $(1, 1)$ to $(3, 3)$ in the maze below using A* with the Manhattan distance as the heuristic, avoiding cyclical paths. The allowed actions consist of moving to one of the four adjacent squares, unless blocked by a wall. In case of tie choose the node with the lowest path cost, then the shallowest node, and finally a random node.



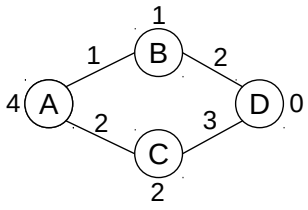
5. The search tree shown below has been obtained for some search problem after the first three expansion steps. The letters inside each node denote the corresponding states; operator costs are shown on the edges; the value of an admissible heuristic function are shown next to each node. (a) Was this search tree generated by A*? (clearly motivate your answer) (b) If your answer to the previous question is "yes," specify in which order the inner nodes have been expanded, and which node A* will expand next.



6. Assume the search tree shown below has been obtained for some search problem after the first three expansion steps by some uninformed strategy. The meaning of letters and numbers is the same as in the previous exercise. Determine whether the search strategy can be depth-first search, uniform cost search, both of them, or none of them, clearly motivating your answer.



7. The state space of some search problem is shown in the figure below. The operator costs are shown on the edges, and the values of a heuristic function, assuming `D` is the goal state, are shown next to each node. Is this heuristic *admissible*? (Clearly motivate your answer.)



Additional questions

(a) When is a search strategy *optimal*? Under what condition is A* optimal?

(b) When is a search strategy *complete*? Is Breadth-first search complete? Why, or why not?

(c) What are the *time* and *space complexity* of graph search algorithms? What is the "unit of measurement" that is used to express them?

(d) Explain why the worst-case *time complexity* of Breadth-first search is $\mathcal{O}(2^{n+1})$, with reference to a hypothetical search problem with constant branching factor $b = 2$ (i.e., each node has exactly two successors), and shallowest solution(s) at depth n.

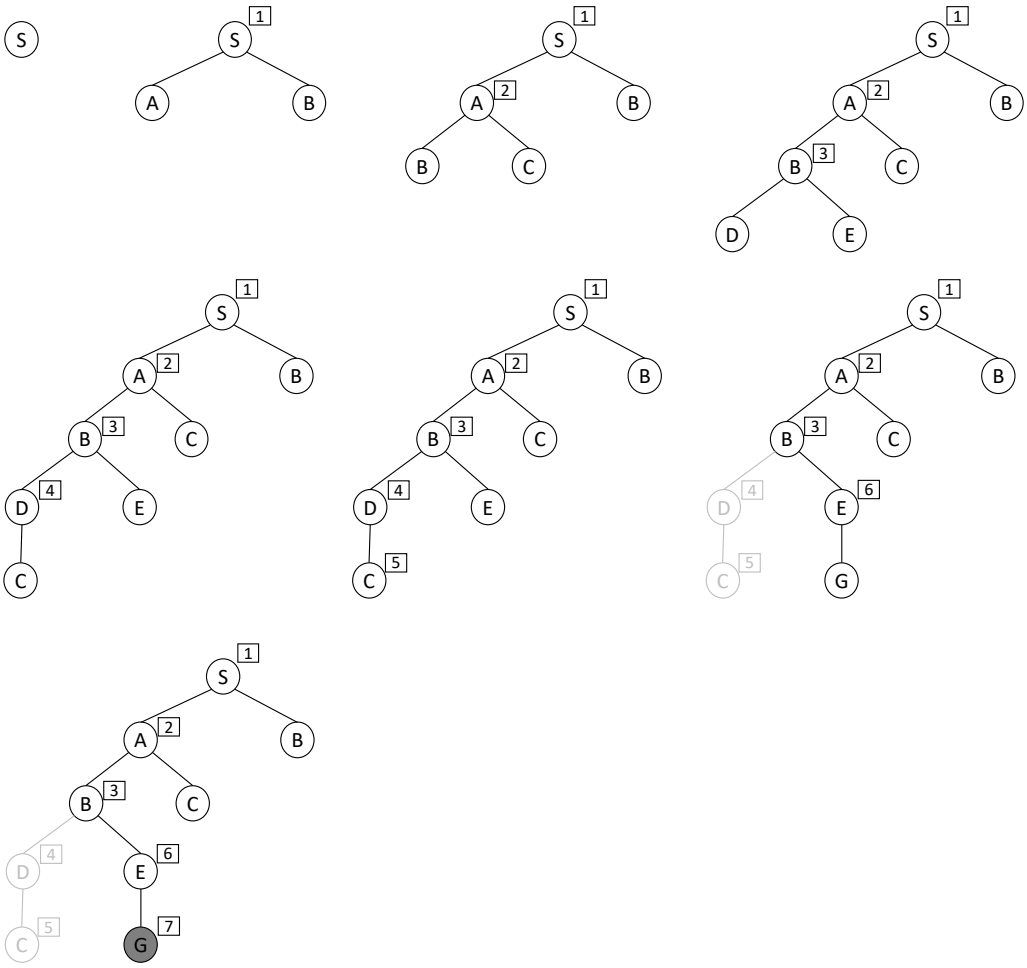
(e) Explain why Depth-first search has $\mathcal{O}(mb)$ worst-case *space complexity*, with reference to a hypothetical problem with constant branching factor b, maximum depth m and all the solutions at the same depth m.

(f) Under what condition is a heuristic function *admissible*? Can you make an example of a non-admissible heuristic for the search problem of exercise 1, by modifying a *single* value in the corresponding table? What is the difference between admissible and non-admissible heuristics for the A* search strategy?

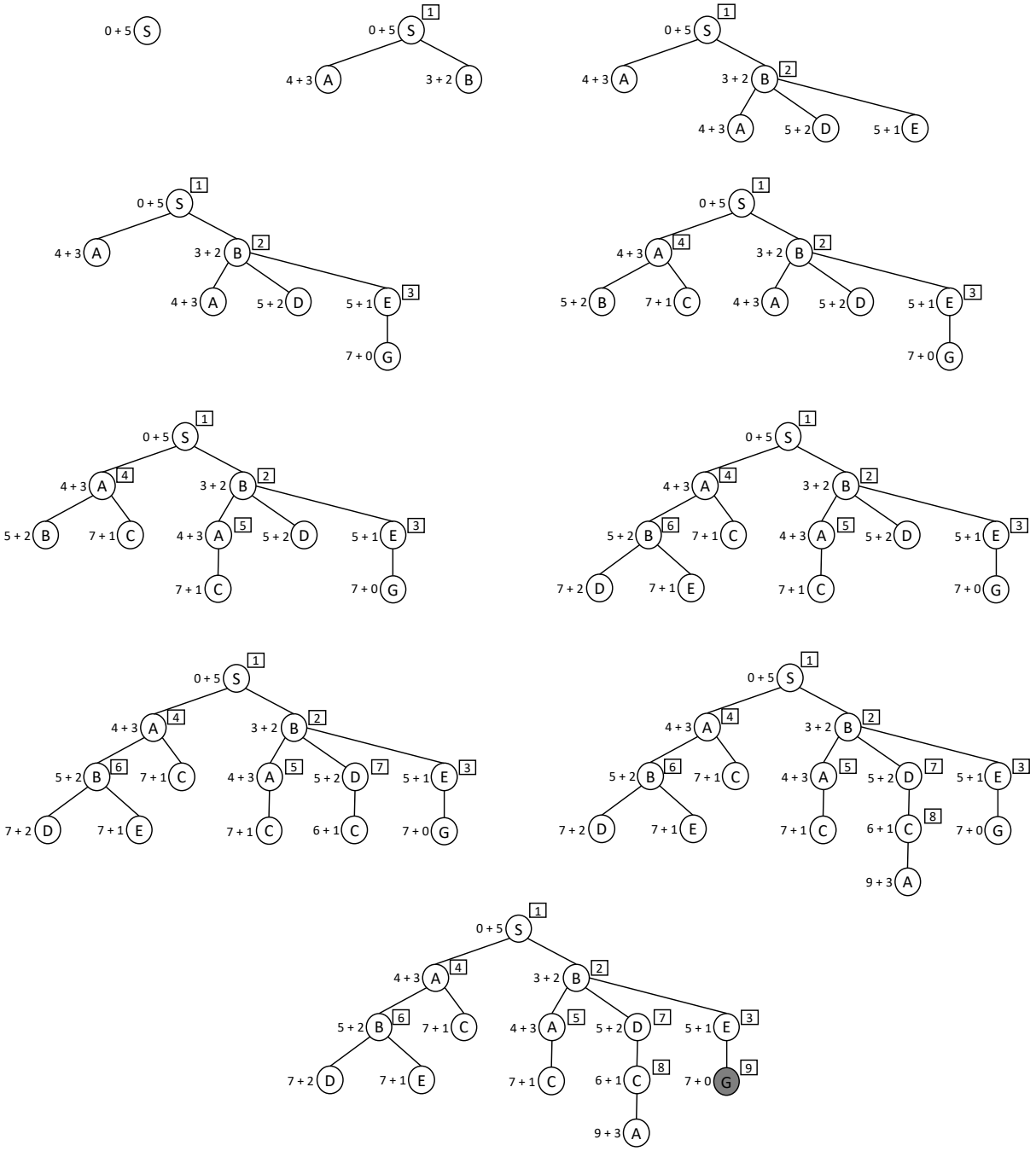
Solution of exercises

Exercise 1

The search tree built by depth-first search is shown below. The expansion steps are indicated by numbers inside squares, next to the corresponding nodes. Note that in step $\boxed{5}$ no successor is found, since all the states reachable from `C` (i.e., `A` and `D`) are already present in the path from the root node; therefore, before proceeding with the next expansion step, the corresponding node is deleted from the search tree, as well as its parent node, since at this point the parent node itself has no successors anymore; deleted nodes are shaded in the next partial trees. Note also that, according to the general search algorithm, a solution is found at step $\boxed{7}$, when a node containing the goal state is *selected* for expansion, not at step $\boxed{6}$, when it is *generated* by expanding its parent node.

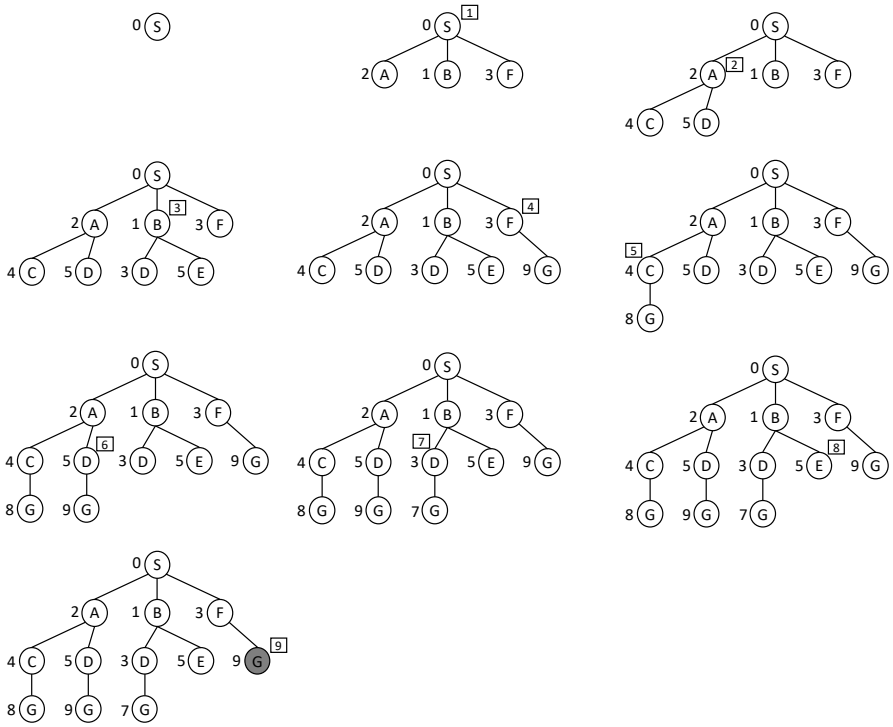


The search tree built by A* is shown below. On the left of each node the sum of the corresponding path cost g and heuristic function h is shown as $g + h$.

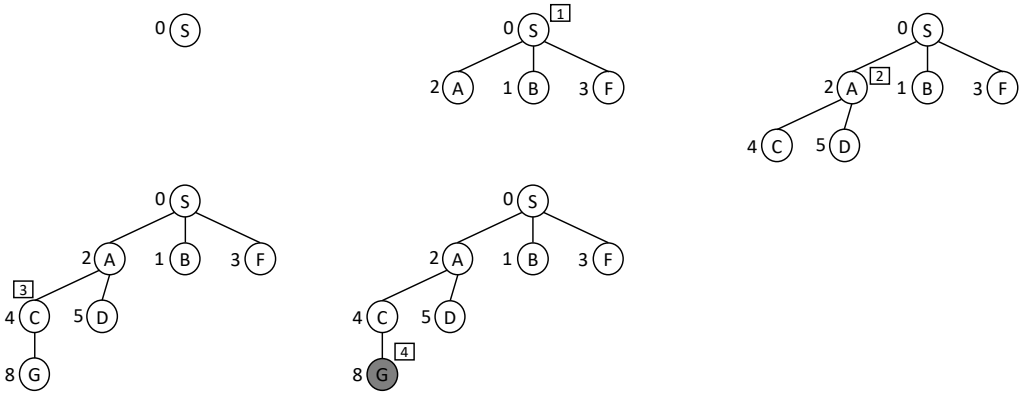


Exercise 2

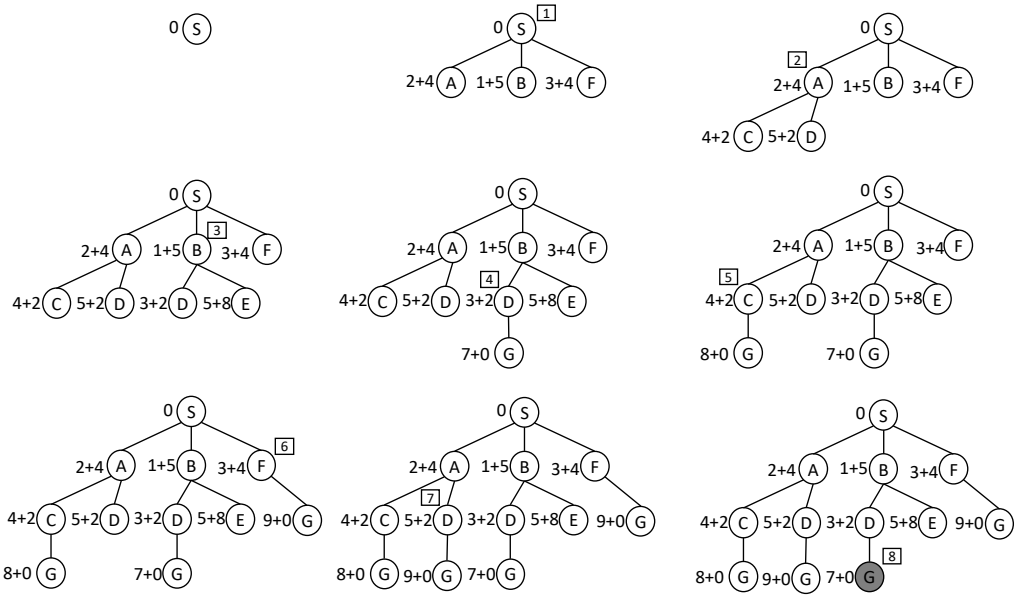
The search tree built by breadth-first search is shown below. The path cost is shown next to each node. Note that in step $\boxed{8}$ the node selected for expansion has no successors. A solution is found at step $\boxed{9}$. Note that it is not the optimal solution (which is not guaranteed to be found by breadth-first search).



The search tree built by depth-first search is shown below. In this case a solution (not the optimal one) is found in the first explored path.

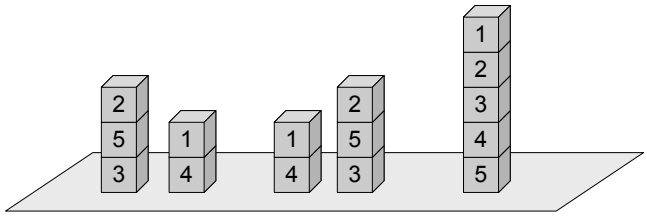


Finally, the search tree built by A* is shown below. Since the heuristic is admissible, the solution found by A* is guaranteed to be optimal. Note that the solutions found by breadth-first and depth-first search (see above) have a higher cost. Note also that after *generating* the optimal solution in step $\boxed{4}$, A* *generates* also a suboptimal one in step $\boxed{7}$: the node expanded at step $\boxed{7}$ has the same evaluation function as the node with the optimal solution, but has a lower path cost, and thus is chosen according to the tiebreaker. Of course, the node with a suboptimal solution cannot be selected later for expansion (and thus cannot be returned as a solution).



Exercise 3

(a) The state space is made up of the set of distinct arrangements of the five blocks into one or more (up to five) piles. A state can be represented by a *set*[1] of piles, and each pile can be represented as a *sequence* of block numbers, e.g., sorted as the blocks of the pile from top to bottom. For instance, the two equivalent configurations of piles (states) shown on the left and in the middle of the figure below are represented by the set $\{(2,5,3),(1,4)\}$, and the goal state on the right is represented by $\{(1,2,3,4,5)\}$.



The operators can be formally described as follows: given a state $\{(b_{1,1},\ldots,b_{1,n_1}),\ldots,(b_{p,1},\ldots,b_{p,n_p})\}$, where p denotes the number of piles ($1 \le p \le 5$) and n_k the number of blocks in the k-th pile ($n_k \ge 1$ for each k, and $\sum_{k=1}^{p} n_k = 5$), the operators consist of moving one of the top blocks $b_{k,1}$ ($k = 1,\ldots,p$) either to the table, thus generating a new pile (only if the original pile contains more than one block, i.e., $n_k > 1$), or on top of one of the other $p-1$ piles (if any).

The operators can be implemented by a *successor function* that receives as an argument the representation s of a state, and returns all the pairs (s', a) where s' is one of the states obtained as

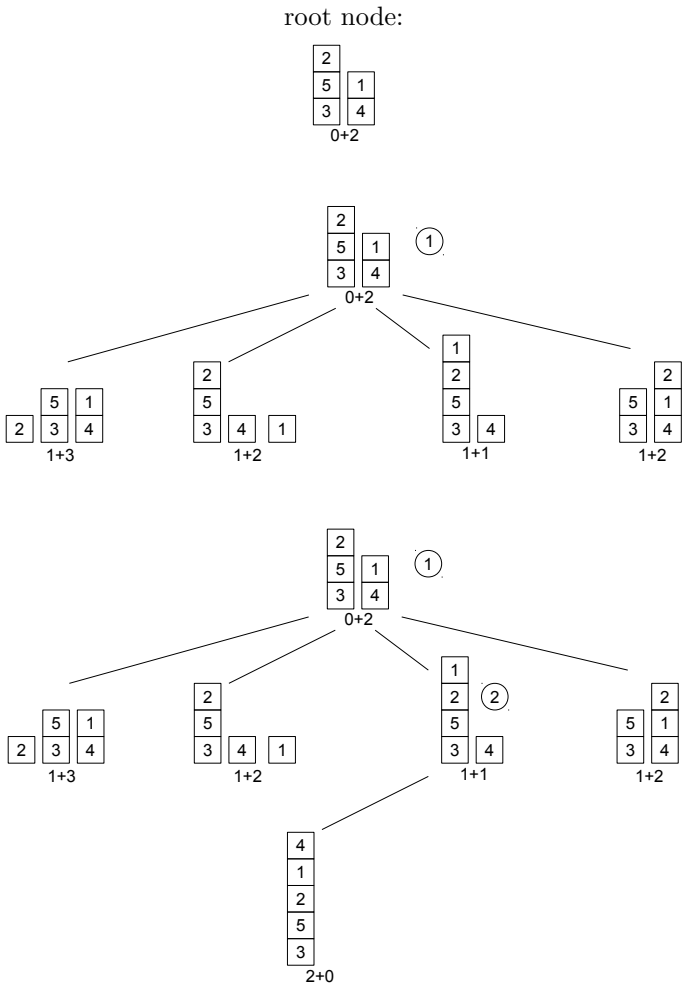
[1]Remember that the ordering between piles is not relevant.

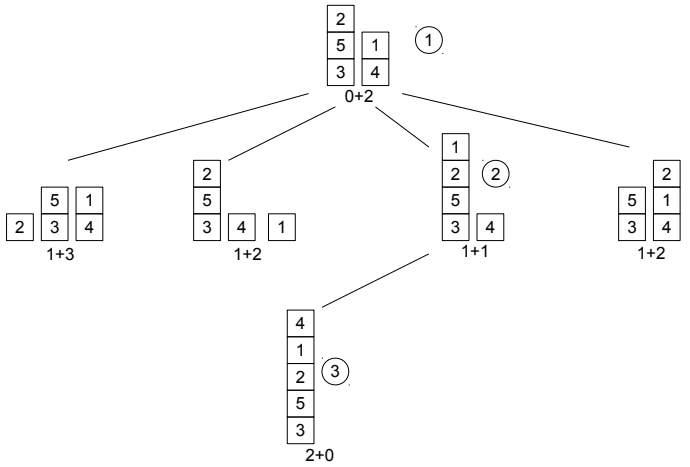
described above, and a the description of the corresponding operator. The description can be a pair (b', b''), where b' is the number of the moved block $b_{k,1}$ and b'' the number of the block on top of which it has been placed, or 0 for the table. For instance, from state $\{(2,5,3),(1,4)\}$ the operator $(2,0)$ consists of moving block 2 to the table, leading to the state $\{(5,3),(2),(1,4)\}$; similarly, the operator $(1,2)$ consists of moving block 1 on top of block 2, leading to the state $\{(1,2,5,3),(4)\}$.

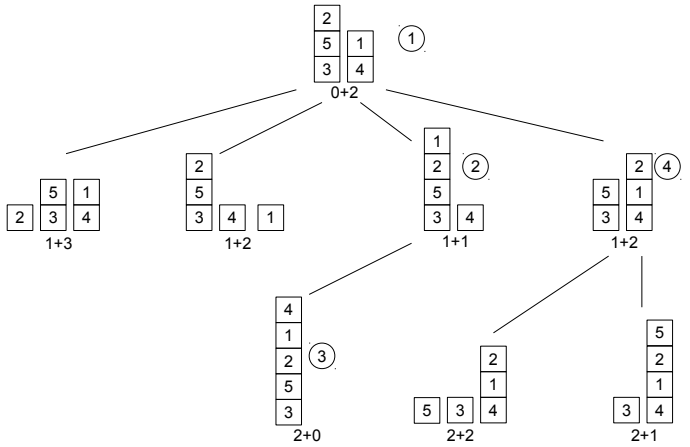
Finally, since the goal is to reach the target configuration by moving the smallest number of blocks, it follows that each operator has a cost of 1.

(b) The search tree obtained by A* after the first four expansion steps, using the heuristic given in the text and avoiding repeated states in the same path from the root, is shown in the figure below. For the sake of clarity, the tree obtained after the expansion of each node is separately shown.

The A* evaluation function (path cost + heuristic) is shown below each node. The order of expansion is shown next to the expanded node. Note that the node selected for expansion at step 3 has no successors. At step 4 the tie between the two nodes with identical evaluation function, path cost and depth is broken randomly. Finally, note that after four expansion steps a solution has not yet been found (not even generated).

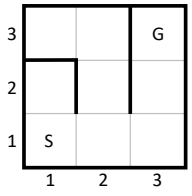


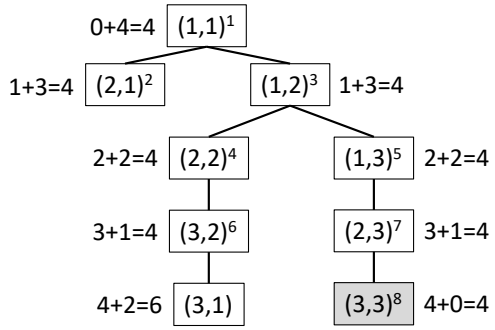




Exercise 4

For convenience the maze is reported below, next to the search tree generated by A*. In the search tree, states are denoted as (row index, column index). The values of the evaluation function (path cost + heuristic) are shown next to each node. The order of expansion is indicated as a superscript on each expanded node. Note that node $(2, 1)$, that is selected for expansion at step 2, has no successors.





Exercise 5

Remember that A* selects for expansion the leaf node n (or one of the leaf nodes) with the lowest value of the evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost and $h(n)$ the heuristic function.

(a) After expanding the root node, the evaluation function of its successors equals $2 + 3 = 5$ for node `B` and $1 + 2 = 3$ for node `C`. A* would therefore expand node `C`. The evaluation function of its successors equals $4 + 3 = 7$ for node `F` and $4 + 1 = 5$ for node `G`. Now there would be a tie between nodes `B` and `G`, which could be broken either randomly or choosing the node with lowest path cost, i.e., `B`. In the former case the resulting search tree *can* be, and in the latter it *is*, the considered one. So the answer is: yes, this search tree can be generated by A*.

(b) The order of expansion is `A`, `C`, `B`. The evaluation function of node `B`'s successors equals $3 + 2 = 4$ for node `D` and $6 + 0 = 6$ for node `E` (by the way, this is a goal node, since its heuristic function equals 0). The node that will be expanded next by A* is therefore `D`.

Exercise 6

The search strategy used to obtain the considered search tree cannot be depth-first: indeed, after the expansion of the root node, either node `C` or node `D` has been expanded in the second step; in the former case either node `E` or node `F` would have been expanded in the third step, instead of node `D`; in the latter case either node `G` or node `H` would have been expanded in the third step, instead of node `C`.

The search strategy cannot be uniform-cost, either. After the expansion of the root node, the path cost of its successors `B`, `C` and `D` is 9, 7 and 3, respectively; uniform-cost search would therefore expand node `D`. In turn, the path cost of its successors `G` and `H` is $3 + 3 = 6$ and $3 + 5 = 8$, respectively; at this step uniform-cost search would expand node `G`, instead of node `C`.

Exercise 7

A heuristic function is admissible if it never overestimates the minimum path cost from a state to the goal state. It is easy to see that this condition is violated by the considered heuristic. Indeed, the minimum-cost path from state `A` to `D` is `A` $\rightarrow$ `B` $\rightarrow$ `D`, whose cost is 3, whereas the heuristic value of state `A` is 4, which *overestimates* the minimum path cost.